



Jun 30, 2026

Testing Report

Key Insights From TestSprite's Autonomous AI Testing Agent

Report Contents

1	Executive Summary	03
2	Frontend UI Test Results	03

21

Test Runs

42.9%

Pass Rate

12

Issues

2h

Saved By TestSprite

Table of Contents

- 3 Executive Summary
 - 3 High-Level Overview
 - 3 Key Findings

- 3 Frontend UI Test Results
 - 3 Test Coverage Summary
 - 3 Test Execution Summary
 - 5 Test Execution Breakdown

Executive Summary

1 High-Level Overview

OVERVIEW	
Total APIs Tested	0 APIs
Base URL	https://nuxt-app-q3vg.vercel.app/
Pass / Fail / Blocked	Backend (endpoint): 0 Passed / 0 Failed Frontend: 9 Passed / 12 Failed

2 Key Findings

Test Summary

This frontend run is mixed: 6 tests passed and 9 failed, with no blocked cases. The executable subset score is based on a 40% success rate, which indicates the app is functional for some core journeys but unstable across several important user flows. Positive signs include passing authentication gating, login, browsing, and empty-state coverage. However, failures cluster around navigation and account/cart-related interactions, suggesting the deployed UI is not consistently exposing expected controls or states. That pattern points to reliability issues that directly affect user experience on the hosted build.

What could be better

The main weakness is UI accessibility and runtime stability in the deployed frontend. Multiple tests time out waiting for expected controls such as `Home`, `Cart`, `View All`, and `Profile Details`, and one click fails because the `Electronic` nav link becomes unstable and detaches from the DOM. Several assertions also land on icon-only spans instead of text, such as `Auto-logout in:` and `0 results found for "nonexistent-product-xyz"`, which suggests markup or hydration regressions in production. The failures are concentrated in navigation, cart, and account settings, rather than being isolated to one feature.

Recommendations

Prioritize the header/nav and account/cart views in the deployed Nuxt app. First, verify the production build and hydration for the top navigation so `Home`, `Cart`, `View All`, and category links render as stable, accessible controls. Then inspect the account settings and product-search empty states to ensure the expected text nodes are present instead of icon-only spans. After each fix, rerun the affected Playwright scenarios to confirm the selectors and visible copy match the tests. Use the fix prompt below as the direct worklist.

Frontend UI Test Results

1 Test Coverage Summary

This report summarizes the frontend UI testing results for the application. TestSprite's AI agent automatically generated and executed tests based on the UI structure, user interaction flows, and visual components. The tests aimed to validate core functionalities, visual correctness, and responsiveness across different states.

URL NAME	TEST CASES	PASS/FAIL RATE
nuxt-app-q3vg.vercel.app	21	9 Pass / 12 Fail

Note

The test cases were generated using real-time analysis of the application's UI hierarchy and user flows. Some visual and functional validations were adapted dynamically based on runtime DOM changes.

2 Test Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
Orders: Open an order from order history	Verify that a customer can open an order detail view from the orders list. The expected outcome is that order-specific information becomes visible.	Medium	Failed
Authentication gating: Block access to the cart when signed out	Verify that an unauthenticated visitor cannot open the cart page. The expected outcome is that the cart is protected and requires sign in.	Low	Passed
Homepage browsing: Browse categories from the homepage	Verify that a shopper can open the storefront, explore categories, and reach product listings from the homepage. The flow should expose discoverable products for browsing and selection.	High	Passed
Orders: View order history	Verify that a returning customer can open the orders page and review past purchases. The expected outcome is that the order list is visible.	Medium	Passed
Cart persistence: Keep cart items after navigating away	Verify that selected cart items persist when the shopper moves between browsing pages and the cart. The expected outcome is that the cart contents remain intact across navigation.	Medium	Failed
Login: Sign in to access the customer account	Verify that a returning customer can sign in through the login page and reach authenticated account features. The expected outcome is successful access to the user account area.	High	Passed
Account settings: View account profile and saved addresses	Verify that an authenticated customer can open account settings and review profile information and saved addresses. The expected outcome is that account details are visible and accessible from the dashboard.	Medium	Failed
Session management: Show a logout timer in account settings	Verify that account settings includes a visible session timer for authenticated users. The expected outcome is that the timer is present on the account page.	Low	Failed
Homepage browsing: Discover products from the homepage collections	Verify that a shopper can use the homepage to move from curated collections into product discovery. The expected outcome is that products become visible and available for selection.	High	Passed
Checkout: Proceed from cart to checkout entry	Verify that a shopper can move from the cart into the checkout entry flow. The expected outcome is that the checkout action is available from the cart page.	High	Failed
Product catalog: Add a filtered electronic product to the cart	Verify that a shopper can narrow the electronic catalog and add a matching item to the cart. The expected outcome is that the cart reflects the product chosen from the filtered results.	High	Failed
Product catalog: Browse electronic products with filters	Verify that a shopper can open the electronic catalog, refine products with category, search, and price controls, and see matching results. The expected outcome is that filtered products remain visible for further shopping.	Medium	Failed
Authentication gating: Block access to orders when signed out	Verify that an unauthenticated visitor cannot open the orders page. The expected outcome is that the orders area is protected and requires sign in.	Low	Passed
Authentication gating: Block access to account settings when signed out	Verify that an unauthenticated visitor cannot open the account settings page. The expected outcome is that access is gated and the user is directed to sign in.	Low	Passed
Cart: Review items in the cart	Verify that a shopper can open the cart and review the selected items before checkout. The expected outcome is that item details and the checkout option are visible.	High	Failed
Orders: Show an empty orders state	Verify that a customer with no purchase history sees an empty orders page. The expected outcome is a clear empty state instead of a broken order list.	Low	Passed
Login validation: Reject invalid login credentials	Verify that the login form blocks access when invalid credentials are entered. The expected outcome is that an error indication appears and the user is not signed in.	Medium	Passed
Product catalog: Reject an invalid product search and filter combination	Verify that the electronic catalog handles a search and filter combination that returns no matches. The expected outcome is an empty state rather than misleading product results.	Low	Failed
Cart: View empty cart state	Verify that an authenticated customer can open the cart when no items have been selected. The expected outcome is an empty cart state with no mistaken purchase	Low	Failed

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
	items.		
Session management: Log out from account settings	Verify that a signed-in customer can end their session from account settings. The expected outcome is that the user is returned to a signed-out state and can no longer access the account page directly.	Medium	Failed
Add to cart: Add a product to the cart from the homepage	Verify that a shopper can browse from the homepage and add an item to the cart. The expected outcome is that the selected product is reflected in the shopping cart state.	High	Failed

3 Test Execution Breakdown

Nuxt-App-Q3vg.Vercel.App Failed & Blocked Test Details

Orders: Open an order from order history

ATTRIBUTES			
Status	Failed		
Priority	Medium		
Description	Verify that a customer can open an order detail view from the orders list. The expected outcome is that order-specific information becomes visible.		
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/1782749252578624/tmp/510dcd92-d29d-4447-8799-333fe5ad4652/result.webm		

```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Open the Login page so the sign-in form can be filled
48         # (navigate to the site's Login page).
49         await page.goto("https://nuxt-app-q3vg.vercel.app/login")
50         try:
51             await page.wait_for_load_state("domcontentloaded",
52                 timeout=5000)
53         except Exception:
54             pass
55
56         # -> Fill the Email field with 'user@yopmail.com', fill the
57         # Password field with 'Dipu1234@', then click the 'Sign In'
58         # button to sign in.
```

```

51     # Email address email field
52     elem = page.get_by_placeholder('Email address', exact=True)
53     await elem.wait_for(state="visible", timeout=10000)
54     await elem.fill("user@yopmail.com")
55
56     # -> Fill the Email field with 'user@yopmail.com', fill the
    Password field with 'Dipu1234@', then click the 'Sign In'
    button to sign in.
57     # Password password field
58     elem = page.get_by_placeholder('Password', exact=True)
59     await elem.wait_for(state="visible", timeout=10000)
60     await elem.fill("Dipu1234@")
61
62     # -> Fill the Email field with 'user@yopmail.com', fill the
    Password field with 'Dipu1234@', then click the 'Sign In'
    button to sign in.
63     # Sign In button
64     elem = page.get_by_role('button', name='Sign In', exact=True)
65     await elem.click(timeout=10000)
66
67     # -> Click the 'View All' button in the Recent Orders card
    to open the Orders page and display the full orders list.
68     # View All button
69     elem = page.get_by_role('button', name='View All',
    exact=True)
70     await elem.click(timeout=10000)
71
72     # -> Click the first order's right-side 'open order' button
    (the circular arrow) to open the order detail view and
    verify order-specific information is displayed.
73     # Click the first order's right-side 'open order' button
    (the circular arrow) to open the order detail view and
    verify order-specific information is displayed.
74     elem = page.locator('xpath=/html/body/div/div/div[2]/main/
    div/div[3]/div[2]/main/div/div[3]/div/div/div[2]/div[2]/
    span')
75     await elem.click(timeout=10000)
76
77     # --> Assertions to verify final state
78
79     # --> Verify order details are displayed
80     # Assert: Order subtotal '₹799' is visible in the order
    details.
81     await expect(page.locator("xpath=/html/body/div[6]/div/div
    [2]/div/div[2]/div[3]/div/div[1]/span[2]").nth(0)).
    to_have_text("\u20b9799", timeout=15000), "Order subtotal
    '\u20b9799' is visible in the order details."
82     # Assert: The 'Total' label is visible in the order details.
83     await expect(page.locator("xpath=/html/body/div[6]/div/div
    [2]/div/div[2]/div[3]/div/div[3]/span[1]").nth(0)).
    to_have_text("Total", timeout=15000), "The 'Total' label is
    visible in the order details."
84     # Assert: The 'Close' button is visible in the order details
    modal.
85     await expect(page.locator("xpath=/html/body/div[6]/div/div
    [2]/div/div[2]/div[4]/button[1]").nth(0)).to_have_text
    ("Close", timeout=15000), "The 'Close' button is visible in
    the order details modal."
86     # Assert: The 'Track Package' button is visible in the order

```



```

    details modal.
87     await expect(page.locator("xpath=/html/body/div[6]/div/div
    [2]/div/div[2]/div[4]/button[2]").nth(0)).to_have_text
    ("Track Package", timeout=15000), "The 'Track Package'
    button is visible in the order details modal."
88     await asyncio.sleep(5)
89
90     finally:
91         if context:
92             await context.close()
93         if browser:
94             await browser.close()
95         if pw:
96             await pw.stop()
97
98     asyncio.run(run_test())
99

```

Error

TimeoutError: Locator.click: Timeout 10000ms exceeded. Call log: - waiting for get_by_role("button", name="View All", exact=True)

Cause

The hosted page likely does not expose an accessible button named 'View All' at runtime, so the test waits until timeout. This can happen if the element was removed/renamed during deployment, rendered as a non-button element without the expected role/text, hidden behind a loading/error state, or never mounted because of a client-side render/hydration failure or delayed data dependency on the host. In short, the deployed service is not presenting the expected CTA in time or at all.

Fix

Ensure the homepage actually renders a button with accessible role=button and exact accessible name 'View All' within 10 seconds. If the UI text or element type changed, update the deployed app to restore that CTA (or make it a real <button> with visible text 'View All'). Also verify the page is not blocked by runtime errors, hydration failures, or slow client-side data fetching that prevent the button from appearing; check Vercel logs/build output and fix any failing API calls, missing env vars, or client-side exceptions so the initial content loads reliably.

Cart persistence: Keep cart items after navigating away

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	Verify that selected cart items persist when the shopper moves between browsing pages and the cart. The expected outcome is that the cart contents remain intact across navigation.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/1782749576270639//tmp/1cadaa6f-1cf3-4dc8-b1ec-5696d92387ee/result.webm

```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Click the product's 'Add to Cart' button to add one
48         # item, then open the Cart page and verify the selected item
49         # remains in the cart.
50         # Add to Cart button
51         elem = page.get_by_text('Celling Fan', exact=True).locator(
52             "xpath=ancestor-or-self::*[.//button][1]").get_by_role(
53             'button', name='Add to Cart', exact=True)
54         await elem.click(timeout=10000)
55
56         # -> Click the product's 'Add to Cart' button to add one
57         # item, then open the Cart page and verify the selected item
58         # remains in the cart.
```

```

49     await page.goto("https://nuxt-app-q3vg.vercel.app/user/cart")
50     try:
51         await page.wait_for_load_state("domcontentloaded",
52                                     timeout=5000)
53     except Exception:
54         pass
55
56     # -> Navigate to the storefront homepage (D-Shop) to locate
57     # product listings and 'Add to Cart' buttons so a product can
58     # be added.
59     await page.goto("https://nuxt-app-q3vg.vercel.app/")
60     try:
61         await page.wait_for_load_state("domcontentloaded",
62                                     timeout=5000)
63     except Exception:
64         pass
65
66     # -> Click the 'Add to Cart' button for the 'Celling Fan'
67     # product on the homepage.
68     # Add to Cart button
69     elem = page.get_by_text('Celling Fan', exact=True).locator
70     ("xpath=ancestor-or-self::*[./button][1]").get_by_role
71     ('button', name='Add to Cart', exact=True)
72     await elem.click(timeout=10000)
73
74     # -> Click the 'Add to Cart' button for the 'Celling Fan'
75     # product on the homepage.
76     # Electronic link
77     elem = page.locator("xpath=/html/body/div[1]/div/div[2]/
78     header/header/nav/div[2]/ul/li[2]/a").nth(0)
79     await elem.click(timeout=10000)
80
81     # -> Click the 'Add to Cart' button for the 'Celling Fan'
82     # product on the homepage.
83     # Cart link
84     elem = page.get_by_role('link', name='Cart', exact=True)
85     await elem.click(timeout=10000)
86
87     # -> Sign in using the 'Email address' and 'Password' fields
88     # and click the 'Sign In' button to authenticate the shopper.
89     # Email address email field
90     elem = page.get_by_placeholder('Email address', exact=True)
91     await elem.wait_for(state="visible", timeout=10000)
92     await elem.fill("user@yopmail.com")
93
94     # -> Sign in using the 'Email address' and 'Password' fields
95     # and click the 'Sign In' button to authenticate the shopper.
96     # Password password field
97     elem = page.get_by_placeholder('Password', exact=True)
98     await elem.wait_for(state="visible", timeout=10000)
99     await elem.fill("Dipu1234@")
100
101     # -> Sign in using the 'Email address' and 'Password' fields
102     # and click the 'Sign In' button to authenticate the shopper.
103     # Sign In button
104     elem = page.get_by_role('button', name='Sign In', exact=True)
105     await elem.click(timeout=10000)
106
107     # -> Click the 'Home' link in the header to open the

```

```

storefront homepage so a product can be added to the cart.
95     # Home link
96     elem = page.get_by_role('link', name='Home', exact=True)
97     await elem.click(timeout=10000)
98
99     # -> Click the 'Add to Cart' (product action) for the
    'Celling Fan' product on the homepage, then navigate to the
    'Electronic' category page.
100    # In Cart button
101    elem = page.get_by_text('Celling Fan', exact=True).locator
    ("xpath=ancestor-or-self::*[.//button][1]").get_by_role
    ('button', name='In Cart', exact=True)
102    await elem.click(timeout=10000)
103
104    # -> Click the 'Add to Cart' (product action) for the
    'Celling Fan' product on the homepage, then navigate to the
    'Electronic' category page.
105    # Electronic link
106    elem = page.get_by_role('link', name='Electronic',
    exact=True)
107    await elem.click(timeout=10000)
108
109    # -> Click the 'Add to Cart' button on the 'Celling Fan'
    product card, then open the Cart by clicking the Cart icon
    in the header and verify the cart lists the 'Celling Fan'.
110    # Add to Cart button
111    elem = page.get_by_text('Electronics', exact=True).locator
    ("xpath=ancestor-or-self::*[.//button][1]").get_by_role
    ('button', name='Add to Cart', exact=True)
112    await elem.click(timeout=10000)
113
114    # -> Click the 'Add to Cart' button on the 'Celling Fan'
    product card, then open the Cart by clicking the Cart icon
    in the header and verify the cart lists the 'Celling Fan'.
115    # Cart link
116    elem = page.get_by_role('link', name='Cart', exact=True)
117    await elem.click(timeout=10000)
118
119    # -> Click the 'Home' link in the header to navigate to the
    storefront homepage (simulate leaving the Cart), then after
    the homepage loads return to the Cart to verify the 'Celling
    Fan' remains in the cart.
120    # Home link
121    elem = page.get_by_role('link', name='Home', exact=True)
122    await elem.click(timeout=10000)
123
124    # -> Open the 'Cart' by clicking the Cart icon in the
    header, then verify the cart still lists the 'Celling Fan'
    item.
125    # Cart link
126    elem = page.get_by_role('link', name='Cart', exact=True)
127    await elem.click(timeout=10000)
128
129    # --> Assertions to verify final state
130
131    # --> Verify the cart still contains the selected item
132    # Assert: The cart lists the 'Celling Fan' item, confirming
    it persisted.
133    await expect(page.locator("xpath=/html/body/div[1]/div/div

```

```

[2]/main/div/div[3]/div[2]/div[1]/div[1]/div[1]/span").nth
(0)).to_contain_text("Celling Fan", timeout=15000), "The
cart lists the 'Celling Fan' item, confirming it persisted."
134     await asyncio.sleep(5)
135
136     finally:
137         if context:
138             await context.close()
139         if browser:
140             await browser.close()
141         if pw:
142             await pw.stop()
143
144     asyncio.run(run_test())
145

```

Error

TimeoutError: Locator.click: Timeout 10000ms exceeded. Call log: - waiting for locator("xpath=/html/body/div[1]/div/div[2]/header/header/nav/div[2]/ul/li[2]/a").first - locator resolved to <a data-v-8f71cd7b="" href="/products?category=electronic" class="md:p-4 py-2 block text-text/70 hover:text-primary hover:-translate-y-0.5 transition-all font-medium">Electronic - attempting click action - waiting for element to be visible, enabled and stable - element is not stable - retrying click action - waiting for element to be visible, enabled and stable - element was detached from the DOM, retrying

Cause

The test found the Electronic nav link, but it was not stable and then detached from the DOM while Playwright attempted to click it. This usually means the hosting build is causing a hydration mismatch or a client-side re-render/animation in the header after load, which remounts the link or shifts the layout. The result is a transient element that cannot be clicked reliably within the timeout.

Fix

The click target is being re-rendered or moved during interaction, likely because the header/nav is inside a client-side responsive menu/animation and the DOM updates after hydration. On hosting, fix by making the navigation stable: avoid replacing the <a> node during hover/menu transitions, disable layout-shifting animations on the header/menu during initial load, and ensure SSR/CSR markup matches so hydration does not remount the nav. Also verify there is no auto-opening/closing mobile menu or breakpoint-dependent script running on desktop that reflows the nav when the page is first interacted with.

Account settings: View account profile and saved addresses

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	Verify that an authenticated customer can open account settings and review profile information and saved addresses. The expected outcome is that account details are visible and accessible from the dashboard.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/1782749289464288//tmp/1a2de4d0-cac5-4561-98bc-4f1925d18464/result.webm

```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Click the 'Sign In' button in the header to open the
48         # login page.
49         # Sign In link
50         elem = page.get_by_role('link', name='Sign In', exact=True)
51         await elem.click(timeout=10000)
52
53         # -> input
54         # Email address email field
55         elem = page.get_by_placeholder('Email address', exact=True)
56         await elem.wait_for(state="visible", timeout=10000)
57         await elem.fill("user@yopmail.com")
58     
```



```

54     # -> input
55     # Password password field
56     elem = page.get_by_placeholder('Password', exact=True)
57     await elem.wait_for(state="visible", timeout=10000)
58     await elem.fill("Dipu1234@")
59
60     # -> click
61     # Sign In button
62     elem = page.get_by_role('button', name='Sign In', exact=True)
63     await elem.click(timeout=10000)
64
65     # -> Click the 'Profile Details' button in the left menu to
        open the profile information section and verify that profile
        fields (for example full name and email) are displayed.
66     # Profile Details button
67     elem = page.get_by_role('button', name='Profile Details',
        exact=True)
68     await elem.click(timeout=10000)
69
70     # -> Click the 'Saved Addresses' button in the left menu to
        open the Saved Addresses panel and verify any saved shipping
        addresses are displayed.
71     # Saved Addresses button
72     elem = page.get_by_role('button', name='Saved Addresses',
        exact=True)
73     await elem.click(timeout=10000)
74
75     # --> Assertions to verify final state
76
77     # --> Verify profile information is displayed
78     # Assert: The user's full name 'Dipu Sharma' is visible in
        the profile sidebar.
79     await expect(page.locator("xpath=/html/body/div[1]/div/div
        [2]/main/div/div[3]/div[2]/aside/div/div/div/div[2]/span").
        nth(0)).to_contain_text("Dipu Sharma", timeout=15000), "The
        user's full name 'Dipu Sharma' is visible in the profile
        sidebar."
80     # Assert: The user's email 'user@yopmail.com' is visible in
        the profile sidebar.
81     await expect(page.locator("xpath=/html/body/div[1]/div/div
        [2]/main/div/div[3]/div[2]/aside/div/div/div/div[2]/span").
        nth(0)).to_contain_text("user@yopmail.com", timeout=15000),
        "The user's email 'user@yopmail.com' is visible in the
        profile sidebar."
82     await page.locator("xpath=/html/body/div[1]/div/div[2]/main/
        div/div[3]/div[2]/aside/div/div/div/div[1]/img").nth(0).
        scroll_into_view_if_needed()
83     # Assert: The profile avatar image is visible.
84     await expect(page.locator("xpath=/html/body/div[1]/div/div
        [2]/main/div/div[3]/div[2]/aside/div/div/div/div[1]/img").nth
        (0)).to_be_visible(timeout=15000), "The profile avatar image
        is visible."
85
86     # --> Verify saved addresses are displayed
87     # Assert: Saved Addresses header 'Shipping Addresses' is
        visible.
88     await expect(page.locator("xpath=/html/body/div[1]/div/div
        [2]/main/div/div[3]/div[2]/main/div/div[1]/div/h2/span").nth
        (0)).to_contain_text("Shipping Addresses", timeout=15000),

```

```

89         "Saved Addresses header 'Shipping Addresses' is visible."
90         # Assert: The 'Add New Address' button is visible in the
          Saved Addresses section.
91         await expect(page.locator("xpath=/html/body/div[1]/div/div
          [2]/main/div/div[3]/div[2]/main/div/div[1]/button").nth(0)).
          to_have_text("Add New Address", timeout=15000), "The 'Add
          New Address' button is visible in the Saved Addresses
          section."
92         await page.locator("xpath=/html/body/div[1]/div/div[2]/main/
          div/div[3]/div[2]/main/div/div[2]/div/div/div/button").nth
          (0).scroll_into_view_if_needed()
93         # Assert: At least one saved address card is visible.
94         await expect(page.locator("xpath=/html/body/div[1]/div/div
          [2]/main/div/div[3]/div[2]/main/div/div[2]/div/div/div/
          button").nth(0)).to_be_visible(timeout=15000), "At least one
          saved address card is visible."
95         await asyncio.sleep(5)
96     finally:
97         if context:
98             await context.close()
99         if browser:
100             await browser.close()
101         if pw:
102             await pw.stop()
103
104     asyncio.run(run_test())
105

```

Error

TimeoutError: Locator.click: Timeout 10000ms exceeded. Call log: - waiting for get_by_role("button", name="Profile Details", exact=True)

Cause

The hosted page likely does not expose an accessible button named "Profile Details" within 10 seconds, either because the component is not rendered on the deployed route, is blocked by a client-side rendering/hydration issue, is hidden behind auth/loading state, or the element is present but not semantically a button with the expected accessible name. A deployment mismatch or runtime error could also prevent the UI from reaching the expected state.

Fix

Ensure the page renders a clickable element with role=button and exact accessible name "Profile Details" on initial load. If the control is conditionally rendered or depends on client-side hydration/data, make sure it appears without requiring delayed API calls, hydration blockers, or route guards. Verify that the deployed page matches the tested route, that no A/B or auth gating hides the button, and that SSR/hydration errors are not preventing the component from mounting. If necessary, fix the component markup to use an actual <button> (or proper aria role/name), and check for CSS overlays or disabled states that may make it unavailable to Playwright.

Session management: Show a logout timer in account settings

ATTRIBUTES	
Status	Failed
Priority	Low
Description	Verify that account settings includes a visible session timer for authenticated users. The expected outcome is that the timer is present on the account page.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/178274918864907///tmp/9b73c951-0557-477a-bef9-b0fe6f3afb24/result.webm

```
1 import asyncio
2 import re
3 from playwright import async_api
4 from playwright.async_api import expect
5
6 async def run_test():
7     pw = None
8     browser = None
9     context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Click the 'Sign In' link in the header to open the
48         # login page.
49         # Sign In link
50         elem = page.get_by_role('link', name='Sign In', exact=True)
51         await elem.click(timeout=10000)
52
53         # -> Fill 'user@yopmail.com' into the Email address field,
54         # fill 'Dipu1234@' into the Password field, then click the
55         # 'Sign In' button to submit the form.
56         # Email address email field
57         elem = page.get_by_placeholder('Email address', exact=True)
58         await elem.wait_for(state="visible", timeout=10000)
```

```

52     await elem.fill("user@yopmail.com")
53
54     # -> Fill 'user@yopmail.com' into the Email address field,
    fill 'Dipu1234@' into the Password field, then click the
    'Sign In' button to submit the form.
55     # Password password field
56     elem = page.get_by_placeholder('Password', exact=True)
57     await elem.wait_for(state="visible", timeout=10000)
58     await elem.fill("Dipu1234@")
59
60     # -> Fill 'user@yopmail.com' into the Email address field,
    fill 'Dipu1234@' into the Password field, then click the
    'Sign In' button to submit the form.
61     # Sign In button
62     elem = page.get_by_role('button', name='Sign In', exact=True)
63     await elem.click(timeout=10000)
64
65     # --> Assertions to verify final state
66
67     # --> Verify a logout timer is displayed
68     # Assert: The account settings page displays the logout
    timer.
69     await expect(page.locator("xpath=/html/body/div[1]/div/div
    [2]/main/div/div[3]/div[2]/main/div/div/div[1]/div/div[1]/
    span").nth(0)).to_contain_text("Auto-logout in:",
    timeout=15000), "The account settings page displays the
    logout timer."
70     await asyncio.sleep(5)
71
72     finally:
73         if context:
74             await context.close()
75         if browser:
76             await browser.close()
77         if pw:
78             await pw.stop()
79
80     asyncio.run(run_test())
81

```

Error

Locator expected to contain text 'Auto-logout in:' Actual value: Call log: - Expect "to_contain_text" with timeout 15000ms
 - waiting for locator("xpath=/html/body/div[1]/div/div[2]/main/div/div[3]/div[2]/main/div/div/div[1]/div/div[1]/span").first
 10 × locator resolved to
 - unexpected value ""

Cause

The hosted app's rendered HTML differs from what the test expects: the locator resolves to an icon-only rather than the text container, so the 'Auto-logout in:' label is either missing, moved, or hidden due to a UI/layout regression in the deployed build. This could be caused by a component refactor, wrong build artifact deployed, a conditional render failing in production, or a CSS/icon wrapper taking the place of the text element.

Fix

Check the deployed page's DOM structure and the auto-logout widget implementation. The test is selecting the first span inside the card area and expects it to contain the label text, but the host is rendering an icon span there instead. Update the production component so the visible text node for the auto-logout status is present in the expected location (or restore the intended markup/order), and ensure the deploy picked up the latest build. If the content is conditionally rendered, verify the feature flag/state on the hosted environment is enabled and the correct data is being loaded so 'Auto-logout in:' appears in that section.

Checkout: Proceed from cart to checkout entry

ATTRIBUTES	
Status	Failed
Priority	High
Description	Verify that a shopper can move from the cart into the checkout entry flow. The expected outcome is that the checkout action is available from the cart page.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/1782749317625516//tmp/7e12c0cb-7cca-48d8-9081-614fc04a2cb1/result.webm

```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Click the 'Add to Cart' button for the first product
48         # (Celling Fan) to add an item to the cart.
49         # Add to Cart button
50         elem = page.get_by_text('Celling Fan', exact=True).locator
51             ("xpath=ancestor-or-self::*[.//button][1]").get_by_role
52             ('button', name='Add to Cart', exact=True)
53         await elem.click(timeout=10000)
54
55         # -> Open the site's Cart page (Cart) to inspect cart
56         # contents and look for a visible Checkout action or button.
57         await page.goto("https://nuxt-app-q3vg.vercel.app/user/cart")
58         try:
```



```

51         await page.wait_for_load_state("domcontentloaded",
52             timeout=5000)
53     except Exception:
54         pass
55
56     # -> Sign in using the email 'user@yopmail.com' and password
57     # 'Dipu1234@' by filling the email and password fields and
58     # clicking the 'Sign In' button.
59     # Email address email field
60     elem = page.get_by_placeholder('Email address', exact=True)
61     await elem.wait_for(state="visible", timeout=10000)
62     await elem.fill("user@yopmail.com")
63
64     # -> Sign in using the email 'user@yopmail.com' and password
65     # 'Dipu1234@' by filling the email and password fields and
66     # clicking the 'Sign In' button.
67     # Password password field
68     elem = page.get_by_placeholder('Password', exact=True)
69     await elem.wait_for(state="visible", timeout=10000)
70     await elem.fill("Dipu1234@")
71
72     # -> Sign in using the email 'user@yopmail.com' and password
73     # 'Dipu1234@' by filling the email and password fields and
74     # clicking the 'Sign In' button.
75     # Sign In button
76     elem = page.get_by_role('button', name='Sign In', exact=True)
77     await elem.click(timeout=10000)
78
79     # -> Click the 'Cart' icon in the header to open the Cart
80     # page and check for a 'Checkout' or 'Proceed to Checkout'
81     # button.
82     # Cart link
83     elem = page.get_by_role('link', name='Cart', exact=True)
84     await elem.click(timeout=10000)
85
86     # -> Click the 'Proceed to Checkout' button in the Order
87     # Summary to enter the checkout flow and verify the checkout
88     # page or checkout form appears.
89     # Proceed to Checkout button
90     elem = page.get_by_role('button', name='Proceed to
91     Checkout', exact=True)
92     await elem.click(timeout=10000)
93
94     # --> Assertions to verify final state
95
96     # --> Verify the checkout entry flow is reached
97     # Assert: The 'Secure Checkout' modal title is visible,
98     # indicating the checkout flow was reached.
99     await expect(page.locator("xpath=/html/body/div[5]/div/div
100     [2]/div/h2/span").nth(0)).to_have_text("Secure Checkout",
101     timeout=15000), "The 'Secure Checkout' modal title is
102     visible, indicating the checkout flow was reached."
103
104     # Assert: The checkout modal shows the 'Cash on Delivery'
105     # payment option.
106     await expect(page.locator("xpath=/html/body/div[5]/div/div
107     [2]/div/div[1]/div[2]/div/button[1]").nth(0)).to_have_text
108     ("Cash on Delivery", timeout=15000), "The checkout modal
109     shows the 'Cash on Delivery' payment option."
110
111     # Assert: The checkout modal contains the 'Confirm & Pay'

```

```

button with the order amount.
90     await expect(page.locator("xpath=/html/body/div[5]/div/div
[2]/div/div[2]/button[2]").nth(0)).to_have_text("Confirm &
Pay \u20b92,118.25", timeout=15000), "The checkout modal
contains the 'Confirm & Pay' button with the order amount."
91     await asyncio.sleep(5)
92
93     finally:
94         if context:
95             await context.close()
96         if browser:
97             await browser.close()
98         if pw:
99             await pw.stop()
100
101     asyncio.run(run_test())
102

```

Error

TimeoutError: Locator.click: Timeout 10000ms exceeded. Call log: - waiting for get_by_role("link", name="Cart", exact=True)

Cause

The production site likely does not expose the expected 'Cart' navigation link in the DOM when the test runs, so Playwright times out waiting for `get_by_role('link', name='Cart', exact=True)`. This can happen if the link is conditionally rendered only after client-side hydration, hidden on certain breakpoints, blocked by a runtime error, or changed in production due to a build/config mismatch on the hosting side.

Fix

Ensure the header/navigation renders a visible, accessible link with `role='link'` and exact text 'Cart' on initial load in production. Check for SSR/client hydration issues, route guards, conditional rendering by viewport/auth state, or styling that hides/disables the Cart link on Vercel. Verify the deployed build includes the expected navigation markup and that no runtime errors prevent the header from mounting; redeploy after fixing any environment-specific config or missing data causing the Cart link to not appear.

Product catalog: Add a filtered electronic product to the cart

ATTRIBUTES	
Status	Failed
Priority	High
Description	Verify that a shopper can narrow the electronic catalog and add a matching item to the cart. The expected outcome is that the cart reflects the product chosen from the filtered results.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/1782749539126356//tmp/48ec865a-e91a-4838-8b4b-e88ec56aac9d/result.webm

```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Click the 'Electronic' link in the top navigation to
48         # open the electronic products catalog.
49         # Electronic link
50         elem = page.get_by_role('link', name='Electronic',
51             exact=True)
52         await elem.click(timeout=10000)
53
54         # -> Select the 'Electronics' category filter, enter 'Fan'
55         # into the search field, click 'SEARCH', and click 'Apply
56         # Price' to set a valid price range (this narrows results
57         # before adding to cart).
58         # Electronics button
```

```

50     elem = page.get_by_text('Categories', exact=True).locator
      ("xpath=ancestor-or-self::*[.//button][1]").get_by_role
      ('button', name='Electronics', exact=True)
51     await elem.click(timeout=10000)
52
53     # -> Select the 'Electronics' category filter, enter 'Fan'
      into the search field, click 'SEARCH', and click 'Apply
      Price' to set a valid price range (this narrows results
      before adding to cart).
54     # Search our collection... text field
55     elem = page.get_by_placeholder('Search our collection...',
      exact=True)
56     await elem.wait_for(state="visible", timeout=10000)
57     await elem.fill("Fan")
58
59     # -> Select the 'Electronics' category filter, enter 'Fan'
      into the search field, click 'SEARCH', and click 'Apply
      Price' to set a valid price range (this narrows results
      before adding to cart).
60     # SEARCH button
61     elem = page.get_by_role('button', name='SEARCH', exact=True)
62     await elem.click(timeout=10000)
63
64     # -> Select the 'Electronics' category filter, enter 'Fan'
      into the search field, click 'SEARCH', and click 'Apply
      Price' to set a valid price range (this narrows results
      before adding to cart).
65     # Apply Price button
66     elem = page.get_by_role('button', name='Apply Price',
      exact=True)
67     await elem.click(timeout=10000)
68
69     # -> Clear the search field labeled 'Search our collection...
      ' and click the 'SEARCH' button to display Electronics
      products.
70     # Search our collection... text field
71     elem = page.get_by_placeholder('Search our collection...',
      exact=True)
72     await elem.wait_for(state="visible", timeout=10000)
73     await elem.fill("")
74
75     # -> Clear the search field labeled 'Search our collection...
      ' and click the 'SEARCH' button to display Electronics
      products.
76     # SEARCH button
77     elem = page.get_by_role('button', name='SEARCH', exact=True)
78     await elem.click(timeout=10000)
79
80     # -> Click the 'Clear Filters' button to reset all filters
      and reveal available Electronics products.
81     # Clear Filters button
82     elem = page.get_by_role('button', name='Clear Filters',
      exact=True)
83     await elem.click(timeout=10000)
84
85     # -> Click the 'ADD TO CART' button on the 'Celling Fan'
      product card, then open the Cart using the cart icon/link
      and verify the cart lists the added item.
86     # Add to Cart button

```

```

87     elem = page.get_by_text('Electronics', exact=True).locator
      ("xpath=ancestor-or-self::*[.//button][1]").get_by_role
      ('button', name='Add to Cart', exact=True)
88     await elem.click(timeout=10000)
89
90     # -> Click the 'ADD TO CART' button on the 'Celling Fan'
      product card, then open the Cart using the cart icon/link
      and verify the cart lists the added item.
91     # Cart link
92     elem = page.locator("xpath=/html/body/div[1]/div/div[2]/
      header/header/nav/div[2]/ul/li[6]/a").nth(0)
93     await elem.click(timeout=10000)
94
95     # -> Fill the Email field with 'user@yopmail.com', fill the
      Password field with 'Dipu1234@', and click the 'Sign In'
      button to authenticate so the cart can be checked.
96     # Email address email field
97     elem = page.get_by_placeholder('Email address', exact=True)
98     await elem.wait_for(state="visible", timeout=10000)
99     await elem.fill("user@yopmail.com")
100
101    # -> Fill the Email field with 'user@yopmail.com', fill the
      Password field with 'Dipu1234@', and click the 'Sign In'
      button to authenticate so the cart can be checked.
102    # Password password field
103    elem = page.get_by_placeholder('Password', exact=True)
104    await elem.wait_for(state="visible", timeout=10000)
105    await elem.fill("Dipu1234@")
106
107    # -> Fill the Email field with 'user@yopmail.com', fill the
      Password field with 'Dipu1234@', and click the 'Sign In'
      button to authenticate so the cart can be checked.
108    # Sign In button
109    elem = page.get_by_role('button', name='Sign In', exact=True)
110    await elem.click(timeout=10000)
111
112    # -> Open the cart by clicking the 'Cart' icon in the header
      to view cart contents and verify that 'Celling Fan' is
      listed.
113    # Cart link
114    elem = page.get_by_role('link', name='Cart', exact=True)
115    await elem.click(timeout=10000)
116
117    # --> Assertions to verify final state
118
119    # --> Verify the cart reflects the added item
120    # Assert: Expected cart icon badge to show 1 item.
121    await expect(page.locator("xpath=/html/body/div[1]/div/div
      [2]/header/header/nav/div[2]/ul/li[6]/a/span[2]").nth(0)).
      to_have_text("1", timeout=15000), "Expected cart icon badge
      to show 1 item."
122    # Assert: Expected 'T-Shirt, cotton anthetic design' to not
      be visible in the cart.
123    await expect(page.locator("xpath=/html/body/div[1]/div/div
      [2]/main/div/div[3]/div[2]/div[1]/div[2]/div[1]/span").nth
      (0)).not_to_be_visible(timeout=15000), "Expected 'T-Shirt,
      cotton anthetic design' to not be visible in the cart."
124    await asyncio.sleep(5)
125

```

```
126         finally:
127             if context:
128                 await context.close()
129             if browser:
130                 await browser.close()
131             if pw:
132                 await pw.stop()
133
134     asyncio.run(run_test())
135
```

Error

TimeoutError: Locator.click: Timeout 10000ms exceeded. Call log: - waiting for get_by_text("Electronics", exact=True).locator("xpath=ancestor-or-self::*[.//button][1]").get_by_role("button", name="Add to Cart", exact=True)

Cause

The hosting-side failure is likely that the deployed page does not expose a clickable "Add to Cart" button for the Electronics item within the expected card, or it is not becoming interactive in time. Common causes include a hydration/runtime mismatch in the Nuxt app on Vercel, stale/incorrect production data, a loading state that never resolves, or a UI overlay/positioning issue that intercepts clicks or changes the DOM structure so the locator cannot find a clickable button.

Fix

Ensure the Electronics product card renders an enabled, visible "Add to Cart" button in the DOM within the expected card wrapper and at the expected timing. If the button is conditionally rendered, delayed by client-side hydration, or covered by an overlay, fix the Nuxt page so the category section and its controls are fully interactive after load (e.g., avoid hydration mismatches, remove blocking overlays/spinners, and confirm the button is inside the same ancestor card structure the test targets). Also verify the deployed build is serving the correct data for the Electronics category and that no CSS or layout issue is preventing the button from being clickable.

Product catalog: Browse electronic products with filters

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	Verify that a shopper can open the electronic catalog, refine products with category, search, and price controls, and see matching results. The expected outcome is that filtered products remain visible for further shopping.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/178274926263491//tmp/af637664-edb6-4173-b5ee-312ffd304dd7/result.webm


```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Click the 'Electronic' link in the top navigation to
48         # open the electronic products catalog and wait for the page
49         # to load.
50         # Electronic link
51         elem = page.get_by_role('link', name='Electronic',
52             exact=True)
53         await elem.click(timeout=10000)
54
55         # -> Click the 'Electronics' category filter, enter 'phone'
56         # in the search field, click the 'SEARCH' button, then click
57         # 'Apply Price' to enforce the price filter so matching
58         # product cards can be verified.
```

```

49         # Electronics button
50         elem = page.get_by_text('Categories', exact=True).locator(
51             ("xpath=ancestor-or-self::*[.//button][1]").get_by_role
52             ('button', name='Electronics', exact=True)
53         )
54         await elem.click(timeout=10000)
55
56         # -> Click the 'Electronics' category filter, enter 'phone'
57         # in the search field, click the 'SEARCH' button, then click
58         # 'Apply Price' to enforce the price filter so matching
59         # product cards can be verified.
60
61         # Search our collection... text field
62         elem = page.get_by_placeholder('Search our collection...',
63             exact=True)
64         await elem.wait_for(state="visible", timeout=10000)
65         await elem.fill("phone")
66
67         # -> Click the 'Electronics' category filter, enter 'phone'
68         # in the search field, click the 'SEARCH' button, then click
69         # 'Apply Price' to enforce the price filter so matching
70         # product cards can be verified.
71
72         # SEARCH button
73         elem = page.get_by_role('button', name='SEARCH', exact=True)
74         await elem.click(timeout=10000)
75
76         # -> Click the 'Electronics' category filter, enter 'phone'
77         # in the search field, click the 'SEARCH' button, then click
78         # 'Apply Price' to enforce the price filter so matching
79         # product cards can be verified.
80
81         # Apply Price button
82         elem = page.get_by_role('button', name='Apply Price',
83             exact=True)
84         await elem.click(timeout=10000)
85
86         # --> Assertions to verify final state
87         # Assert: Verify matching product cards are displayed
88         assert False, "Expected: Verify matching product cards are
89         displayed (could not be verified on the page)"
90         await asyncio.sleep(5)
91
92     finally:
93         if context:
94             await context.close()
95         if browser:
96             await browser.close()
97         if pw:
98             await pw.stop()
99
100     asyncio.run(run_test())
101
102

```

Error

TimeoutError: Locator.click: Timeout 10000ms exceeded. Call log: - waiting for get_by_text("Categories", exact=True).locator("xpath=ancestor-or-self::*[.//button][1]").get_by_role("button", name="Electronics", exact=True)

Cause

On the hosted production site, the Categories dropdown likely does not open or does not expose the "Electronics" button in a clickable state within the test timeout. This can happen due to a hydration mismatch or runtime JS error in the deployed Nuxt app, a failed API/data fetch causing the category list to be missing, or CSS/overlay issues in production (for example the dropdown being rendered offscreen, hidden, or covered). As a result, Playwright waits for the Electronics button to become clickable and times out.

Fix

Ensure the Categories menu renders and is interactive on the hosted production build. Verify that the dropdown/popover containing the "Electronics" option is mounted in the DOM after clicking "Categories", is not hidden behind an overlay, and is not blocked by CSS/SSR hydration issues or client-side JS errors. Check that the Nuxt/Vercel production build is shipping the required component and state data, that hydration completes correctly, and that no runtime error prevents the menu from opening. If the menu is populated from an API, confirm the request succeeds in production and the response includes the Electronics category. Also review z-index/pointer-events/scroll locking so the button is actually clickable in the deployed environment.

Cart: Review items in the cart

ATTRIBUTES	
Status	Failed
Priority	High
Description	Verify that a shopper can open the cart and review the selected items before checkout. The expected outcome is that item details and the checkout option are visible.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/1782749405529657//tmp/3baa14e4-9ec5-4f52-9544-0e3e85544ae1/result.webm

```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Click the 'Add to Cart' button for the visible product
48         # labeled 'Add to Cart' under the 'Celling Fan' product to add
49         # it to the shopping cart.
50         # Add to Cart button
51         elem = page.get_by_text('Celling Fan', exact=True).locator(
52             "xpath=ancestor-or-self::*[.//button][1]").get_by_role(
53             'button', name='Add to Cart', exact=True)
54         await elem.click(timeout=10000)
55
56         # -> Open the Cart page by navigating to the site's Cart
57         # page and confirm the added item's details and that a
58         # checkout option/button is shown.
```

```

49     await page.goto("https://nuxt-app-q3vg.vercel.app/user/cart")
50     try:
51         await page.wait_for_load_state("domcontentloaded",
52                                     timeout=5000)
53     except Exception:
54         pass
55
56     # -> Fill the Email and Password fields on the sign-in page
57     # and click the 'Sign In' button to authenticate the shopper
58     # (use email 'user@yopmail.com' and password 'Dipu1234@').
59     # Email address email field
60     elem = page.get_by_placeholder('Email address', exact=True)
61     await elem.wait_for(state="visible", timeout=10000)
62     await elem.fill("user@yopmail.com")
63
64     # -> Fill the Email and Password fields on the sign-in page
65     # and click the 'Sign In' button to authenticate the shopper
66     # (use email 'user@yopmail.com' and password 'Dipu1234@').
67     # Password password field
68     elem = page.get_by_placeholder('Password', exact=True)
69     await elem.wait_for(state="visible", timeout=10000)
70     await elem.fill("Dipu1234@")
71
72     # -> Fill the Email and Password fields on the sign-in page
73     # and click the 'Sign In' button to authenticate the shopper
74     # (use email 'user@yopmail.com' and password 'Dipu1234@').
75     # Sign In button
76     elem = page.get_by_role('button', name='Sign In', exact=True)
77     await elem.click(timeout=10000)
78
79     # -> Click the 'Sign In' button on the login page to
80     # authenticate the user so the cart page can be accessed and
81     # verified.
82     # Sign In button
83     elem = page.get_by_role('button', name='Sign In', exact=True)
84     await elem.click(timeout=10000)
85
86     # -> Click the 'Cart' icon in the header to open the Cart
87     # page and verify that the selected items (name, price,
88     # quantity) and a checkout button are visible.
89     # Cart link
90     elem = page.get_by_role('link', name='Cart', exact=True)
91     await elem.click(timeout=10000)
92
93     # --> Assertions to verify final state
94
95     # --> Verify the item list is displayed
96     # Assert: The cart displays the item 'Celling Fan' in the
97     # item list.
98     await expect(page.locator("xpath=/html/body/div[1]/div/div
99                               [2]/main/div/div[3]/div[2]/div[1]/div[1]/div[1]/span").nth
100                    (0)).to_have_text("Celling Fan", timeout=15000), "The cart
101                                     displays the item 'Celling Fan' in the item list."
102
103     # Assert: The cart displays the item 'T-Shirt, cotton
104     # anthetic design' in the item list.
105     await expect(page.locator("xpath=/html/body/div[1]/div/div
106                               [2]/main/div/div[3]/div[2]/div[1]/div[2]/div[1]/span").nth
107                    (0)).to_have_text("T-Shirt, cotton anthetic design",
108                                     timeout=15000), "The cart displays the item 'T-Shirt, cotton

```

```

    anthetic design' in the item list."
89
90     # --> Verify the checkout option is displayed
91     await page.locator("xpath=/html/body/div[1]/div/div[2]/main/
div/div[3]/div[2]/div[2]/div/button").nth(0).
scroll_into_view_if_needed()
92     # Assert: The 'Proceed to Checkout' button is visible on the
cart page.
93     await expect(page.locator("xpath=/html/body/div[1]/div/div
[2]/main/div/div[3]/div[2]/div[2]/div/button").nth(0)).
to_be_visible(timeout=15000), "The 'Proceed to Checkout'
button is visible on the cart page."
94     await asyncio.sleep(5)
95
96     finally:
97         if context:
98             await context.close()
99         if browser:
100             await browser.close()
101         if pw:
102             await pw.stop()
103
104     asyncio.run(run_test())
105

```

Error

TimeoutError: Locator.click: Timeout 10000ms exceeded. Call log: - waiting for get_by_role("link", name="Cart", exact=True)

Cause

The test is timing out while looking for an accessible link named exactly "Cart", which suggests the hosted page is not exposing that element in the DOM in time or at all. Likely causes on the hosting side include a client-side rendering/hydration failure, a production-only JavaScript error preventing the navigation from mounting, the cart item being renamed or removed in the deployed version, or CSS/overlay issues making the link inaccessible to Playwright even though the page loads.

Fix

Make sure the site renders a visible, clickable navigation link with accessible role="link" and exact accessible name "Cart" on the initial page load. If the link is generated client-side, ensure the Nuxt app hydrates successfully and the header/nav is not blocked by runtime errors, CSS overlays, auth guards, or responsive breakpoints hiding the element. Verify production deployment includes the correct build assets and no A/B or locale changes renamed the item. If the cart link is supposed to appear only after data fetch, preload the required data or render a fallback so the link exists within the test timeout.

Product catalog: Reject an invalid product search and filter combination

ATTRIBUTES	
Status	Failed
Priority	Low
Description	Verify that the electronic catalog handles a search and filter combination that returns no matches. The expected outcome is an empty state rather than misleading product results.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/1782749237328598//tmp/0ac1ab54-5140-4d06-90f8-c91203d79e62/result.webm


```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Open the Electronic products catalog page (the product
48         # listing for the Electronic category) so the search field and
49         # price filters can be used.
50         await page.goto("https://nuxt-app-q3vg.vercel.app/products?
51             category=electronic")
52         try:
53             await page.wait_for_load_state("domcontentloaded",
54                 timeout=5000)
55         except Exception:
56             pass
57
58         # -> Enter the uncommon search term
```

```

        'nonexistent-product-xyz' into the catalog search field and
        click the 'SEARCH' button to run the search.
51     # Search our collection... text field
52     elem = page.get_by_placeholder('Search our collection...',
        exact=True)
53     await elem.wait_for(state="visible", timeout=10000)
54     await elem.fill("nonexistent-product-xyz")
55
56     # -> Enter the uncommon search term
        'nonexistent-product-xyz' into the catalog search field and
        click the 'SEARCH' button to run the search.
57     # SEARCH button
58     elem = page.get_by_role('button', name='SEARCH', exact=True)
59     await elem.click(timeout=10000)
60
61     # -> Click the 'Apply Price' button to apply the selected
        price range, then verify the page still shows the 'No
        products found' empty-state message.
62     # Apply Price button
63     elem = page.get_by_role('button', name='Apply Price',
        exact=True)
64     await elem.click(timeout=10000)
65
66     # --> Assertions to verify final state
67
68     # --> Verify an empty results state is displayed
69     # Assert: The page displays '0 results found for
        "nonexistent-product-xyz", confirming an empty results
        state.
70     await expect(page.locator("xpath=/html/body/div[1]/div/div
        [2]/main/div/div[4]/div/main/div[2]/div/span").nth(0)).
        to_have_text("0 results found for
        \"nonexistent-product-xyz\"", timeout=15000), "The page
        displays '0 results found for \"nonexistent-product-xyz\"",
        confirming an empty results state."
71     await asyncio.sleep(5)
72
73     finally:
74         if context:
75             await context.close()
76         if browser:
77             await browser.close()
78         if pw:
79             await pw.stop()
80
81     asyncio.run(run_test())
82

```

Error

Locator expected to have text '0 results found for "nonexistent-product-xyz"' Actual value: Call log: - Expect

"to_have_text" with timeout 15000ms - waiting for

locator("xpath=/html/body/div[1]/div/div[2]/main/div/div[4]/div/main/div[2]/div/span").first 12 × locator resolved to

 - unexpected value ""

Cause

The most likely cause is a production-side UI/DOM mismatch on the search results page: the selector targeting the empty-results message is landing on a magnify-close icon span instead of a text-bearing span. This can happen if the hosted app is serving an outdated or misbuilt bundle, a hydration issue swapped the expected text element with an icon-only component, or the component markup changed without matching the test expectations. As a result, the page does not expose the expected '0 results found for "nonexistent-product-xyz"' text at the queried locator.

Fix

The XPath used by the test appears to resolve to an icon element (`<span class="iconify i-mdi:magnify-close ...">`) instead of the results-count text node. On the hosting side, this likely means the rendered DOM structure or CSS/markup differs from what the app/test expects, possibly due to a deployment regression, stale build, or client-side hydration mismatch. Fix by ensuring the search results page renders the expected text element in the tested location for empty results (e.g., an element containing `0 results found for "..."`), and verify the production build includes the correct component/template version. If the icon is intended there, update the test selector; otherwise correct the template so the results message is not replaced/obscured by the icon.

Cart: View empty cart state

ATTRIBUTES	
Status	Failed
Priority	Low
Description	Verify that an authenticated customer can open the cart when no items have been selected. The expected outcome is an empty cart state with no mistaken purchase items.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/178274924533054//tmp/784243ed-fc2f-4c3c-ba7f-c180cb29f559/result.webm

```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Open the site's Login page (the 'Sign In' / Login page)
48         # by navigating to the /login URL so the credentials can be
49         # entered.
50         await page.goto("https://nuxt-app-q3vg.vercel.app/login")
51         try:
52             await page.wait_for_load_state("domcontentloaded",
53                 timeout=5000)
54         except Exception:
55             pass
56
57         # -> Fill the email field with 'user@yopmail.com', fill the
58         # password field with 'Dipu1234@', then click the 'Sign In'
```

```

button to authenticate.
51     # Email address email field
52     elem = page.get_by_placeholder('Email address', exact=True)
53     await elem.wait_for(state="visible", timeout=10000)
54     await elem.fill("user@yopmail.com")
55
56     # -> Fill the email field with 'user@yopmail.com', fill the
password field with 'Dipu1234@', then click the 'Sign In'
button to authenticate.
57     # Password password field
58     elem = page.get_by_placeholder('Password', exact=True)
59     await elem.wait_for(state="visible", timeout=10000)
60     await elem.fill("Dipu1234@")
61
62     # -> Fill the email field with 'user@yopmail.com', fill the
password field with 'Dipu1234@', then click the 'Sign In'
button to authenticate.
63     # Sign In button
64     elem = page.get_by_role('button', name='Sign In', exact=True)
65     await elem.click(timeout=10000)
66
67     # -> Click the 'Cart' icon in the header to open the cart
page and verify it displays an empty cart state (no items
listed or an explicit empty-cart message).
68     # Cart link
69     elem = page.get_by_role('link', name='Cart', exact=True)
70     await elem.click(timeout=10000)
71
72     # --> Assertions to verify final state
73
74     # --> Verify an empty cart state is displayed
75     # Assert: Expected cart icon to show 0 items.
76     await expect(page.locator("xpath=/html/body/div[1]/div/div
[2]/header/header/nav/div[2]/ul/li[6]/a").nth(0)).
to_have_text("0", timeout=15000), "Expected cart icon to
show 0 items."
77     # Assert: Expected 'Proceed to Checkout' button to not be
visible when the cart is empty.
78     await expect(page.locator("xpath=/html/body/div[1]/div/div
[2]/main/div/div[3]/div[2]/div[2]/div/button").nth(0)).
not_to_be_visible(timeout=15000), "Expected 'Proceed to
Checkout' button to not be visible when the cart is empty."
79     # Assert: Expected 'Clear Cart' button to not be visible
when the cart is empty.
80     await expect(page.locator("xpath=/html/body/div[1]/div/div
[2]/main/div/div[3]/div[1]/button").nth(0)).not_to_be_visible
(timeout=15000), "Expected 'Clear Cart' button to not be
visible when the cart is empty."
81     await asyncio.sleep(5)
82
83     finally:
84         if context:
85             await context.close()
86         if browser:
87             await browser.close()
88         if pw:
89             await pw.stop()
90
91     asyncio.run(run_test())

```

Error

TimeoutError: Locator.click: Timeout 10000ms exceeded. Call log: - waiting for get_by_role("link", name="Cart", exact=True)

Cause

The host likely serves a production build where the "Cart" navigation link is missing, not yet hydrated, or inaccessible to assistive selectors at test time. Common hosting-side causes include a deployment mismatch, client-only rendering delay, hydration/runtime error preventing the header from mounting, or CSS/conditional logic making the Cart link unavailable or non-interactive. As a result, Playwright cannot locate a visible exact-match link named "Cart" and times out.

Fix

Ensure the header navigation renders a visible, accessible link with `role="link"` and exact text "Cart" on initial page load on the deployed site. If the Cart item is conditionally rendered (e.g. based on hydration, auth state, viewport, or client-only data), make it available server-side or provide a stable fallback so Playwright can find it within 10s. Also verify the deployed production build isn't failing hydration or hiding the nav via CSS/JS, and that the correct route/component is deployed. If the link is intentionally not a native anchor, add proper ARIA role and accessible name, but prefer using an `<a>` element for the Cart navigation.

Session management: Log out from account settings

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	Verify that a signed-in customer can end their session from account settings. The expected outcome is that the user is returned to a signed-out state and can no longer access the account page directly.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/1782749601231788//tmp/8a3bc610-af33-4126-aa24-a746a6b75592/result.webm


```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Click the 'Sign In' link in the header to open the
48         # Login page.
49         # Sign In link
50         elem = page.get_by_role('link', name='Sign In', exact=True)
51         await elem.click(timeout=10000)
52
53         # -> input
54         # Email address email field
55         elem = page.get_by_placeholder('Email address', exact=True)
56         await elem.wait_for(state="visible", timeout=10000)
57         await elem.fill("user@yopmail.com")
58     
```

```

54     # -> input
55     # Password password field
56     elem = page.get_by_placeholder('Password', exact=True)
57     await elem.wait_for(state="visible", timeout=10000)
58     await elem.fill("Dipu1234@")
59
60     # -> click
61     # Sign In button
62     elem = page.get_by_role('button', name='Sign In', exact=True)
63     await elem.click(timeout=10000)
64
65     # -> Open the profile/menu that contains the account actions
        and locate the 'Log out' or 'Sign out' option so the session
        can be ended.
66     # Profile button
67     elem = page.get_by_role('button', name='Profile', exact=True)
68     await elem.click(timeout=10000)
69
70     # -> Click the 'Logout' button in the profile menu to sign
        out, then verify the app returns to a signed-out state (for
        example the login page appears or /user becomes
        inaccessible).
71     # Logout button
72     elem = page.get_by_role('button', name='Logout', exact=True)
73     await elem.click(timeout=10000)
74
75     # -> Navigate to the Account Settings page (the '/user' URL)
        to confirm that the account page cannot be accessed while
        signed out.
76     await page.goto("https://nuxt-app-q3vg.vercel.app/user")
77     try:
78         await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
79     except Exception:
80         pass
81
82     # -> Navigate to the Account Settings page (the '/user' URL)
        and confirm that it is not accessible by verifying the login
        page is shown or that access is denied.
83     await page.goto("https://nuxt-app-q3vg.vercel.app/user")
84     try:
85         await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
86     except Exception:
87         pass
88
89     # -> Navigate to the Account Settings page (Account
        Settings) and confirm the Sign In page is displayed, proving
        the account page is inaccessible when signed out.
90     await page.goto("https://nuxt-app-q3vg.vercel.app/user")
91     try:
92         await page.wait_for_load_state("domcontentloaded",
            timeout=5000)
93     except Exception:
94         pass
95
96     # -> Navigate to the Account Settings page (https://
nuxt-app-q3vg.vercel.app/user) and verify the Sign In page
        is displayed, proving the account page is inaccessible when

```

```

signed out.
97     await page.goto("https://nuxt-app-q3vg.vercel.app/user")
98     try:
99         await page.wait_for_load_state("domcontentloaded",
100                                     timeout=5000)
101     except Exception:
102         pass
103
104     # -> Navigate to the Account Settings page and confirm the
105     # Sign In page (login form with Email address and Password
106     # fields and 'Sign In' button) is displayed, proving /user is
107     # inaccessible while signed out.
108     await page.goto("https://nuxt-app-q3vg.vercel.app/user")
109     try:
110         await page.wait_for_load_state("domcontentloaded",
111                                     timeout=5000)
112     except Exception:
113         pass
114
115     # -> Navigate to the Account Settings page (https://
116     # nuxt-app-q3vg.vercel.app/user) and verify that the Sign In
117     # page is displayed (look for the Email address and Password
118     # inputs and the 'Sign In' button).
119     await page.goto("https://nuxt-app-q3vg.vercel.app/user")
120     try:
121         await page.wait_for_load_state("domcontentloaded",
122                                     timeout=5000)
123     except Exception:
124         pass
125
126     # -> Navigate to the Account Settings page and verify the
127     # Sign In page (showing the 'Email address' input, the
128     # 'Password' input, and the 'Sign In' button) is displayed,
129     # proving the account page is inaccessible while signed out.
130     await page.goto("https://nuxt-app-q3vg.vercel.app/user")
131     try:
132         await page.wait_for_load_state("domcontentloaded",
133                                     timeout=5000)
134     except Exception:
135         pass
136
137     # --> Assertions to verify final state
138
139     # --> Verify the login page is displayed
140     # Assert: The login page heading contains 'Welcome Back'.
141     await expect(page.locator("xpath=/html/body/div[1]/div/main/
142                               div/div").nth(0)).to_contain_text("Welcome Back",
143                               timeout=15000), "The login page heading contains 'Welcome
144                               Back'."
145
146     await page.locator("xpath=/html/body/div[1]/div/main/div/div/
147                               div[1]/div[2]/form/div[1]/input").nth(0).
148     scroll_into_view_if_needed()
149
150     # Assert: The Email address input is visible on the login
151     # page.
152     await expect(page.locator("xpath=/html/body/div[1]/div/main/
153                               div/div/div[1]/div[2]/form/div[1]/input").nth(0)).
154     to_be_visible(timeout=15000), "The Email address input is
155     visible on the login page."
156
157     await page.locator("xpath=/html/body/div[1]/div/main/div/div/

```

```

        div[1]/div[2]/form/div[2]/input").nth(0).
        scroll_into_view_if_needed()
133     # Assert: The Password input is visible on the login page.
134     await expect(page.locator("xpath=/html/body/div[1]/div/main/
        div/div/div[1]/div[2]/form/div[2]/input").nth(0)).
        to_be_visible(timeout=15000), "The Password input is visible
        on the login page."
135     await page.locator("xpath=/html/body/div[1]/div/main/div/div/
        div[1]/div[2]/form/button").nth(0).scroll_into_view_if_needed
        ()
136     # Assert: The 'Sign In' button is visible on the login page.
137     await expect(page.locator("xpath=/html/body/div[1]/div/main/
        div/div/div[1]/div[2]/form/button").nth(0)).to_be_visible
        (timeout=15000), "The 'Sign In' button is visible on the
        login page."
138     await asyncio.sleep(5)
139
140     finally:
141         if context:
142             await context.close()
143         if browser:
144             await browser.close()
145         if pw:
146             await pw.stop()
147
148     asyncio.run(run_test())
149

```

Error

TimeoutError: Locator.click: Timeout 10000ms exceeded. Call log: - waiting for get_by_role("button", name="Profile", exact=True)

Cause

The hosted app likely did not expose the expected 'Profile' button to the browser within 10 seconds because the production page either failed to hydrate/render that UI, rendered a different accessible name/element, or hid it behind responsive/auth state conditions in the deployed environment. In practice this is commonly caused by a deployment-specific frontend error (missing chunk, runtime exception, asset/API failure, or responsive layout mismatch) rather than a test timing issue.

Fix

Ensure the page actually renders a button with role=button and accessible name exactly 'Profile' in the initial, testable viewport. On the hosting side this may mean fixing a deployment/runtime issue that prevents the navigation/header from hydrating or loading correctly (e.g. failed JS chunk, build error, blocked API call causing the menu to not render, or CSS/media-query logic hiding the button on the deployed environment). Verify production build logs, browser console/network errors, and that the Profile control is present in the DOM with proper semantics after load. If it is in a collapsed menu on small screens, make the menu accessible and visible in the hosted layout or adjust the responsive breakpoint so the test can reach it.

Add to cart: Add a product to the cart from the homepage

ATTRIBUTES	
Status	Failed
Priority	High
Description	Verify that a shopper can browse from the homepage and add an item to the cart. The expected outcome is that the selected product is reflected in the shopping cart state.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b438f448-5041-7015-4bbb-19a762b2905d/1782749324867468//tmp/99bc91da-ac89-46dd-afee-559329a899f5/result.webm

```
1  import asyncio
2  import re
3  from playwright import async_api
4  from playwright.async_api import expect
5
6  async def run_test():
7      pw = None
8      browser = None
9      context = None
10
11     try:
12         # Start a Playwright session in asynchronous mode
13         pw = await async_api.async_playwright().start()
14
15         # Launch a Chromium browser in headless mode with custom
16         # arguments
17         browser = await pw.chromium.launch(
18             headless=True,
19             args=[
20                 "--window-size=1280,720",
21                 "--disable-dev-shm-usage",
22                 "--ipc=host",
23                 "--single-process"
24             ],
25         )
26
27         # Create a new browser context (like an incognito window)
28         context = await browser.new_context()
29         # Wider default timeout to match the agent's DOM-stability
30         # budget;
31         # auto-waiting Playwright APIs (expect, locator.wait_for)
32         # inherit this.
33         context.set_default_timeout(15000)
34
35         # Open a new page in the browser context
36         page = await context.new_page()
37
38         # Interact with the page elements to simulate user flow
39         # -> navigate
40         await page.goto("https://nuxt-app-q3vg.vercel.app/")
41         try:
42             await page.wait_for_load_state("domcontentloaded",
43                 timeout=5000)
44         except Exception:
45             pass
46
47         # -> Click the 'Add to Cart' button on the 'Celling Fan'
48         # product card to add that product to the shopping cart.
49         # Add to Cart button
50         elem = page.get_by_text('Celling Fan', exact=True).locator(
51             "xpath=ancestor-or-self::*[.//button][1]").get_by_role(
52             'button', name='Add to Cart', exact=True)
53         await elem.click(timeout=10000)
54
55         # -> Fill the 'Email address' field with user@yopmail.com,
56         # fill the 'Password' field with Dipu1234@, and click the
57         # 'Sign In' button to authenticate.
58         # Email address email field
```

```

50     elem = page.get_by_placeholder('Email address', exact=True)
51     await elem.wait_for(state="visible", timeout=10000)
52     await elem.fill("user@yopmail.com")
53
54     # -> Fill the 'Email address' field with user@yopmail.com,
    fill the 'Password' field with Dipu1234@, and click the
    'Sign In' button to authenticate.
55     # Password password field
56     elem = page.get_by_placeholder('Password', exact=True)
57     await elem.wait_for(state="visible", timeout=10000)
58     await elem.fill("Dipu1234@")
59
60     # -> Fill the 'Email address' field with user@yopmail.com,
    fill the 'Password' field with Dipu1234@, and click the
    'Sign In' button to authenticate.
61     # Sign In button
62     elem = page.get_by_role('button', name='Sign In', exact=True)
63     await elem.click(timeout=10000)
64
65     # -> Click the 'Home' link in the header to return to the
    storefront homepage so a product card can be selected and
    added to the cart.
66     # Home link
67     elem = page.get_by_role('link', name='Home', exact=True)
68     await elem.click(timeout=10000)
69
70     # -> Click the 'In Cart' button on the 'Celling Fan' product
    card to add that product to the cart, then open the Cart by
    clicking the Cart icon to verify the item appears.
71     # In Cart button
72     elem = page.get_by_text('Celling Fan', exact=True).locator
    ("xpath=ancestor-or-self::*[.//button][1]").get_by_role
    ('button', name='In Cart', exact=True)
73     await elem.click(timeout=10000)
74
75     # -> Click the 'In Cart' button on the 'Celling Fan' product
    card to add that product to the cart, then open the Cart by
    clicking the Cart icon to verify the item appears.
76     # Cart link
77     elem = page.get_by_role('link', name='Cart', exact=True)
78     await elem.click(timeout=10000)
79
80     # --> Assertions to verify final state
81
82     # --> Verify the cart reflects the added item
83     # Assert: The cart contains the product 'Celling Fan'.
84     await expect(page.locator("xpath=/html/body/div[1]/div/div
    [2]/main/div/div[3]/div[2]/div[1]/div[1]/div[1]/span").nth
    (0)).to_have_text("Celling Fan", timeout=15000), "The cart
    contains the product 'Celling Fan'."
85     # Assert: The cart icon badge shows 2 items.
86     await expect(page.locator("xpath=/html/body/div[1]/div/div
    [2]/header/header/nav/div[2]/ul/li[6]/a/span[2]").nth(0)).
    to_have_text("2", timeout=15000), "The cart icon badge shows
    2 items."
87     await asyncio.sleep(5)
88
89     finally:
90         if context:

```

```
91     await context.close()
92     if browser:
93         await browser.close()
94     if pw:
95         await pw.stop()
96
97     asyncio.run(run_test())
98
```

Error

TimeoutError: Locator.click: Timeout 10000ms exceeded. Call log: - waiting for get_by_role("link", name="Home", exact=True)

Cause

The test could not find a clickable accessible link named "Home", which suggests the deployed page either does not render that navigation item, renders it with the wrong element/label, or fails during SSR/hydration so the header/nav is absent on load. On the hosting side this can happen if the production build is broken, environment variables or route data are missing, or client-side errors prevent the navigation from mounting, causing the locator query to time out.

Fix

Ensure the homepage renders an accessible link with role="link" and exact accessible name "Home" on initial load. Check that the navigation component is not hidden behind a client-side hydration error, conditional rendering, or a missing route/data issue on Vercel. If the app uses a custom header/nav, make sure the Home item is present in the server-rendered HTML and not replaced by a button/div, and verify the deployment build succeeds without runtime errors so the expected nav link becomes visible and clickable within the timeout.

